



Chapter 3: Introduction to SQL

Database Principle and Design



Outline

- Overview of The SQL Query Language
- SQL Data Definition
- Basic Query Structure of SQL Queries
- Additional Basic Operations
- Set Operations
- Null Values
- Aggregate Functions
- SELECT Statement Processing Order
- Nested Subqueries
- Modification of the Database



3.1 Overview of The SQL Query Language

- IBM developed the original version of SQL in the early 1970s, called as Sequel.
- Evolved and renamed to Structured Query Language (SQL)
- In 1986, the American National Standards Institute (ANSI) and the International Organization for Standardization (ISO) published 1st SQL standard called SQL-86, followed by SQL-89, SQL-92, SQL:1999, SQL:2003, SQL:2006, SQL:2008, SQL:2011, and SQL:2016.
- Two main parts: DDL, DML
- Commercial systems offer most SQL-92 features, plus varying feature sets from later standards and special proprietary features. So they provide different names for query languages:
 - SQL Server: T-SQL
 - Oracle: PL/SQL
 - MySQL: MySQL
- Not all examples in the textbook may work on your particular system.
- In this course, we will use **T-SQL** as our query language. Therefore, we will mainly talk the SQL features supported by **T-SQL**.



3.2 SQL Data Definition

The SQL data-definition language (DDL) allows the specification of information about relations, including:

- The schema for each relation.
- The type of values associated with each attribute.
- The Integrity constraints.
- The set of indices to be maintained for each relation.
- The views for different purposes.
- Transaction control.
- Security and authorization information for each relation.
- The physical storage structure of each relation on disk.



Create Table

- An SQL relation is defined using the **create table** statement:

create table *r*

$(A_1 D_1, A_2 D_2, \dots, A_n D_n,$
 (integrity-constraint₁),
 ...,
 (integrity-constraint_k));

- *r* is the name of the relation
 - each A_i is an attribute name in the schema of relation *r*
 - D_i is the data type of values in the domain of attribute A_i
 - The semicolon at the end of SQL is optional
- Example:

```
create table instructor (  
    ID            char(5),  
    name         varchar(20),  
    dept_name    varchar(20),  
    salary      numeric(8,2))
```



Domain Types in SQL

- **char(n)**: Fixed length character string, with user-specified length n .
- **varchar(n)**: Variable length character strings, with user-specified maximum length n .
- If we stored a string “Avi” of type **char(10)**, seven spaces are appended to the string to make it 10 characters long.
- In contrast, if we stored “Avi” of type **varchar(10)**, no spaces would be added.
- **Int**: Integer (4 bytes in SQL server).
- **Smallint**: Small integer (2 bytes in SQL server).
- **numeric(p,d)**: Fixed point number (e.g., **numeric(3,1)**, allows 44.5 to be stores exactly, but not 444.5 or 0.32)
- **real**: Floating point number (4 bytes in SQL server).
- **float**. Double-precision floating point numbers (8 bytes in SQL server).

*Although **char** data may waste memory space, DBMS process **char** type data faster than **varchar** data.*



Integrity Constraints in Create Table

- Types of integrity constraints
 - **primary key** ($A_1, \dots, A_n$)
 - **foreign key** ($A_m, \dots, A_n$) **references** r
 - **not null**
- (more integrity constraints in section 4.4)
- Database system prevents any update to the database that violates an integrity constraint.
- Example:

```
create table instructor (  
    ID          char(5),  
    name       varchar(20) not null,  
    dept_name varchar(20),  
    salary     numeric(8,2),  
    primary key (ID),  
    foreign key (dept_name) references department );
```



And a Few More Relation Definitions

- **create table** *student* (
 ID **char**(5),
 name **varchar**(20) not null,
 dept_name **varchar**(20),
 tot_cred **numeric**(3,0),
 primary key (*ID*),
 foreign key (*dept_name*) **references** *department*);
- **create table** *takes* (
 ID **char**(5),
 course_id **varchar**(8),
 sec_id **varchar**(8),
 semester **varchar**(6),
 year **numeric**(4,0),
 grade **varchar**(2),
 primary key (*ID*, *course_id*, *sec_id*, *semester*, *year*) ,
 foreign key (*ID*) **references** *student*,
 foreign key (*course_id*, *sec_id*, *semester*, *year*) **references** *section*);



Modify tables

- **Drop Table**
 - **drop table r**
 - *Delete table r from the database schema.*
- **Alter**
 - **alter table r add A D**
 - where A is the name of the attribute to be added to relation r and D is the domain of A .
 - All exiting tuples in the relation are assigned *null* as the value for the new attribute.
 - **alter table r drop A**
 - where A is the name of an attribute of relation r
 - Dropping of attributes not supported by many databases(e.g, SQL Server).



3.3 Basic Structure of SQL Queries

- A typical SQL query has the form:

select $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where P

(more complex structures will be discussed later)

- A_i represents an attribute
 - R_i represents a relation
 - P is a predicate.
- The result of an SQL query is a relation.



The select Clause

- The **select** clause lists the attributes desired in the result of a query
 - corresponds to the **project** operation of the relational algebra
- Example: find the names of all instructors:

select *name*
from *instructor*
- NOTE: SQL are case **insensitive** (i.e., you may use upper- or lower-case letters.)
 - E.g., *Name* $\equiv$ *NAME* $\equiv$ *name*



The select Clause (Cont.)

- Find the department names of all instructors

select *dept_name*

from *instructor*

- SQL allows duplicates in relations as well as in query results.

<i>dept_name</i>
Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.

- To force the elimination of duplicates, insert the keyword **distinct** after select.

select distinct *dept_name*

from *instructor*



The select Clause (Cont.)

- An asterisk in the select clause denotes “all attributes”

select * from *instructor*

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00



The select Clause (Cont.)

- The **select** clause can contain arithmetic expressions involving the operation, +, −, *, and /, and operating on constants or attributes of tuples.
 - The query:

```
select ID, name, salary/12  
from instructor
```

would return a relation that is the same as the *instructor* relation, except that the value of the attribute *salary* is divided by 12.

ID	name	(No column name)
10101	Srinivasan	5416.666666
12121	Wu	7500.000000
15151	Mozart	3333.333333
22222	Einstein	7916.666666
32343	El Said	5000.000000
33456	Gold	7250.000000
45565	Katz	6250.000000
58583	Califieri	5166.666666
76543	Singh	6666.666666
76766	Crick	6000.000000
83821	Brandt	7666.666666
98345	Kim	6666.666666



The where Clause

- The **where** clause specifies conditions that the result must satisfy
 - Correspond to the selection predicate of the relational algebra.
- To find all instructors in Comp. Sci. dept

```
select name  
from instructor  
where dept_name = 'Comp. Sci.'
```

- SQL allows the use of the logical operators **and**, **or**, and **not**
- The operands of the logical operators can be expressions involving the comparison operators **<**, **<=**, **>**, **>=**, **=**, and **<>**.
- To find all instructors in Comp. Sci. dept with salary > 70000

```
select name  
from instructor  
where dept_name = 'Comp. Sci.' and salary > 70000
```

<i>name</i>
Katz
Brandt



The from Clause

- The **from** clause lists the relations involved in the query
 - Corresponds to the Cartesian product operation of the relational algebra.
- Find the Cartesian product *instructor X teaches*

select *
from instructor, teaches

- generates every possible instructor – teaches pair, with all attributes from both relations.

<i>instructor.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...



Examples

Cartesian product not very useful directly, but useful combined with where-clause condition.

- Find the names of all instructors who have taught some course and the course_id
 - **select** *name, course_id*
from *instructor , teaches*
where *instructor.ID = teaches.ID*
- Find the names of all instructors in the Art department who have taught some course and the course_id
 - **select** *name, course_id*
from *instructor , teaches*
where *instructor.ID = teaches.ID*
and *instructor.dept_name = 'Art'*

<i>name</i>	<i>course_id</i>
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-190
Brandt	CS-319
Kim	EE-181



3.4 Additional Basic Operations



The Rename Operation

- The SQL allows renaming relations and attributes using the **as** clause:
old-name as new-name
- The **as** clause can appear in both the **select** and **from** clauses.



Renaming Attributes

enclosed in
single quotes

- Find the instructor names and their monthly salary:
select name **as** instructor,
 convert (int, salary/12) **as** 'monthly salary',
 dept_name **as** department
from instructor

instructor	monthly salary	department
Srinivasan	5416	Comp. Sci.
Wu	7500	Finance
Mozart	3333	Music
Einstein	7916	Physics
El Said	5000	History
Gold	7250	Physics
Katz	6250	Comp. Sci.
Califieri	5166	History
Singh	6666	Finance
Crick	6000	Biology
Brandt	7666	Comp. Sci.



Renaming Relations

One reason to rename a relation is to replace a long relation name with a shortened version that is more convenient to use elsewhere in the query.

- Example: Find the names of all the student and their advisors:

```
select student.name as student_name, instructor.name as instructor_name
from student, advisor, instructor
where student.ID = s_ID and instructor.ID = i_ID
```

Equivalent to:

```
select S.name as student_name, I.name as instructor_name
from student as S, advisor, instructor as I
where S.ID = s_ID and I.ID = i_ID
```

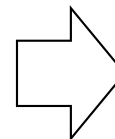
instructor relation

advisor relation

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu		
15151	Mozart		
22222	Einstein		
32343	El Said		
33456	Gold		
45565	Katz		
58583	Califieri		
76543	Singh		
76766	Crick		

student relation

s_id	i_id
00128	45565
12345	10101
23121	76543
44553	22222
45678	22222
76543	45565
76653	98345
98765	98345



student_name	instructor_name
Zhang	Katz
Shankar	Srinivasan
Chavez	Singh
Peltier	Einstein
Levy	Einstein
Brown	Katz
Aoi	Kim
Bourikas	Kim
Tanaka	Crick



Renaming in Self Cartesian Product

- Find the ID and name of those instructors who earn more than the instructor whose ID is 12121

```
select T.ID, T.name  
from instructor as T, instructor as S  
where T.salary > S.salary and S.ID = 12121
```

ID	name
22222	Einstein
83821	Brandt



String Operations

- SQL defines a variety of functions on character strings (e.g, substring, upper, trim). Different database systems may provide different set of string functions. Example:

```
select upper(dept_name) from instructor
```

- SQL specifies strings by enclosing them in single quotes, for example:

```
select name  
from instructor  
where dept_name = 'Comp. Sci.'
```

string

identifiers

- A single quote character that is part of a string can be specified by using two single quote characters; for example, the string “It’s right” can be specified by 'It"s right'.

Note: In SQL standard, the comparison operation on strings is case sensitive (e.g, 'comp. sci.' = 'Comp. Sci.')

However, some database systems, e.g., MySQL and SQL Server, do not distinguish uppercase from lowercase: “'comp. sci.' = 'Comp. Sci.'” would evaluate to true on these systems.



String Operations(cont.)

- SQL includes a **string-matching** operator **like** for comparisons on character strings. The operator **like** uses patterns that are described using two special characters:
 - percent (%). The % character matches any **substring**.
 - underscore (_). The _ character matches any **character**.

- Find the names of all instructors whose name includes the substring “sa”.

```
select name from instructor  
where name like '%sa%'
```

name
Srinivasan
El Said

- Find the names of all instructors whose name are exactly three characters.

```
select name from instructor  
where name like ' _ _ '
```

name
Kim

- Pattern matching examples:
 - 'Intro%' matches any string beginning with “Intro”.
 - '%Comp%' matches any string containing “Comp” as a substring.
 - ' _ _ _ _ %' matches any string of at least four characters.



Ordering the Display of Tuples

- List the names of all instructors in alphabetic order

select *name* **from** *instructor* **order by** *name*

- Can sort on multiple attributes

- Example: **select** * **from** *instructor* **order by** *dept_name*, *name*

ID	name	dept_name	salary
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
45565	Katz	Comp. Sci.	75000.00
10101	Srinivasan	Comp. Sci.	65000.00
98345	Kim	Elec. Eng.	80000.00
76543	Singh	Finance	80000.00
12121	Wu	Finance	90000.00
58583	Califieri	History	62000.00
32343	El Said	History	60000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
33456	Gold	Physics	87000.00

- For each attribute, we may specify **desc** for descending order or **asc** for ascending order (ascending order is the default).

Example: **select** * **from** *instructor* **order by** *dept_name* **asc**, *salary* **desc**



3.5 Set Operations

- Find courses taught in Fall 2017 or in Spring 2018
(**select** *course_id* **from** *section* **where** *semester* = 'Fall' **and** *year* = 2017)
union
(**select** *course_id* **from** *section* **where** *semester* = 'Spring' **and** *year* = 2018)
- Find courses taught in both Fall 2017 and Spring 2018
(**select** *course_id* **from** *section* **where** *semester* = 'Fall' **and** *year* = 2017)
intersect
(**select** *course_id* **from** *section* **where** *semester* = 'Spring' **and** *year* = 2018)
- Find courses taught in Fall 2017 but not in Spring 2018
(**select** *course_id* **from** *section* **where** *semester* = 'Fall' **and** *year* = 2017)
except
(**select** *course_id* **from** *section* **where** *semester* = 'Spring' **and** *year* = 2018)

<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>building</i>	<i>room_number</i>	<i>time_slot_id</i>
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

section relation



3.6 Null Values

- It is possible for tuples to have a null value, denoted by **null**, for some of their attributes
- null** signifies an unknown value or that a value does not exist.

ID	course_id	sec_id	semester	year	grade
44553	PHY-101	1	Fall	2017	B-
45678	CS-101	1	Fall	2017	F
45678	CS-101	1	Spring	2018	B+
45678	CS-319	1	Spring	2018	B
54321	BIO-301	1	Summer	2018	NULL
54321	CS-101	1	Fall	2017	A-
54321	CS-190	2	Spring	2017	B+
55739	MU-199	1	Spring	2018	A-
76543	CS-101	1	Fall	2017	A
76543	CS-319	2	Spring	2018	A
76653	EE-181	1	Spring	2017	C
98765	BIO-301	1	Summer	2018	NULL
98765	CS-101	1	Fall	2017	C-
98765	CS-315	1	Spring	2018	B
98988	BIO-101	1	Summer	2017	A
98988	BIO-301	1	Summer	2018	NULL

takes relation



Null Values (cont.)

- The result of any arithmetic expression involving **null** is **null**
 - Example:
 - **select** *name* ,salary, salary+100 **from** *instructor*

name	salary	(No column name)
Califieri	62000.00	62100.00
Singh	80000.00	80100.00
Crick	72000.00	72100.00
Brandt	92000.00	92100.00
Hong	NULL	NULL
zhong	90000.00	90100.00
Kim	80000.00	80100.00



Null Values (Cont.)

- SQL treats the result of any comparison involving a null value as **unknown**.
 - Example: $5 < \text{null}$ or $\text{null} < \text{null}$ or $\text{null} = \text{null}$
- For Boolean operations in **where** clauses:
 - **and** : $(\text{false and unknown}) = \text{false}$,
 $(\text{true and unknown}) = \text{unknown}$,
 $(\text{unknown and unknown}) = \text{unknown}$
 - **or**: $(\text{unknown or true}) = \text{true}$,
 $(\text{unknown or false}) = \text{unknown}$
 $(\text{unknown or unknown}) = \text{unknown}$
 - **Not**: $\text{not unknown} = \text{unknown}$
- The result of a **where** clause predicate is treated as *false* if it evaluates to *unknown*. Thus the instructor with null salary is not in either results of the following two queries:

`select name, salary from instructor where salary > 2000`
`select name, salary from instructor where not(salary > 2000)`



Null Values (cont.)

- The predicate **is null** and **is not null** can be used to check for null values.
 - Example: Find all students and sections in which they have null grades.

select * from takes where grade is null

- Note that '**is null**' is not identical to '**= null**', and '**is not null**' is not identical to '**≠ null**' either

- Example: Find all instructors whose salary is not null.

select name from instructor where salary is not null



select name from instructor where salary ≠ null





3.7 Aggregate Functions

- Aggregate functions are functions that take **a collection of values** as input and return a single value. SQL offers five standard built-in aggregate functions:
 - avg**: average value
 - min**: minimum value
 - max**: maximum value
 - sum**: sum of values
 - count**: number of values
- Find the average salary of instructors in the Computer Science department
 - **select avg (salary) from instructor where dept_name= 'Comp. Sci.';**
- Find the total number of instructors who teach a course in the Spring 2018 semester
 - **select count (distinct ID) from teaches where semester = 'Spring' and year = 2018;**
- Find the number of tuples in the *course* relation
 - **select count (*) from course;**
- Attributes outside of aggregate functions can not appear in **select** clause (unless they appear in **group by** clause)
 - **select name, min (salary) from instructor**





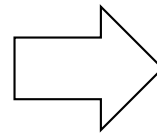
Aggregation with Null values

- In general, all aggregate functions except count (*) ignore null values in their input collection.

- Examples:

**select sum(salary) as sum, count(*) as count, avg(salary) as avg
from instructor where dept_name = 'Comp. Sci.'**

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
45565	Katz	Comp. Sci.	75000.00
83821	Brandt	Comp. Sci.	92000.00
90881	Hong	Comp. Sci.	NULL
90983	zhong	Comp. Sci.	92000.00



sum	count	avg
324000.00	5	81000.000000

$$324000 / 4 = 81000$$



Aggregate Functions – Group By

- Find the average salary of instructors in each department
 - select** *dept_name*, **avg** (*salary*) **as** *avg_salary*
from *instructor*
group by *dept_name*;

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

<i>dept_name</i>	<i>avg_salary</i>
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

select *name*, **min** (*salary*) **from** *instructor*





Aggregation (Cont.)

- Attributes in **select** clause outside of aggregate functions must appear in **group by** list
 - /* erroneous query */
select *dept_name*, *ID*, **avg** (*salary*)
from *instructor*
group by *dept_name*;





Aggregate Functions – Having Clause

- Find the names of all departments whose average salary is greater than 80000

```
select dept_name  
from instructor  
group by dept_name  
having avg (salary) > 80000;
```

- Note:**

- predicates in the **having** clause are applied after the formation of groups whereas predicates in the **where** clause are applied before forming groups

```
select dept_name from instructor  
where avg (salary) > 80000;
```





3.8 SELECT Statement Processing Order

SELECT *column_list*
FROM *table_list*
WHERE *search_condition*
GROUP BY *group_by_list*
HAVING *having_condition*
ORDER BY *order_list* [*ASC|DESC*];

1. The **WHERE** clause excludes rows not meeting its search condition.
2. The **GROUP BY** clause collects the selected rows into groups.
3. Aggregate functions are calculated for each group.
4. The **HAVING** clause further excludes groups not meeting its condition.
5. The **ORDER BY** clause at last sorts the output before it is shown to the user.



3.8 Nested Subqueries

- A **subquery** is a **select-from-where** expression that is nested within another query.
- The nesting can be done in the following SQL query

```
select  $A_1, A_2, \dots, A_n$   
from  $r_1, r_2, \dots, r_m$   
where  $P$ 
```

as follows:

- **Select clause:**
 A_i can be replaced by a subquery that generates a **single** value.
- **From clause:** r_i can be replaced by any valid subquery
- **Where clause:** P can be replaced with an expression of the form:
 $B <\text{operation}> (\text{subquery})$
 B is an attribute and $<\text{operation}>$ to be defined later.



Set Membership

- The **in** operator tests for set membership, where the set is a collection of values produced by a subquery.
- Find courses offered in both Fall 2017 and Spring 2018

```
select distinct course_id from section  
where semester = 'Fall' and year = 2017 and course_id in  
      (select course_id from section where semester = 'Spring' and year = 2018)
```

- Equivalent to:

```
(select course_id from section where semester = 'Fall' and year = 2017)  
intersect  
(select course_id from section where semester = 'Spring' and year = 2018)
```

Note: When this query is written by intersection, **distinct** is unnecessary because the set operation removes duplicates by default.

- Can this query be written like this?

```
select distinct course_id from section  
where (semester = 'Fall' and year = 2017)  
      and (semester = 'Spring' and year = 2018)
```





Set Membership (Cont.)

- Find courses offered in Fall 2017 but not in Spring 2018

```
select distinct course_id from section
where semester = 'Fall' and year = 2017 and
       course_id not in (select course_id from section
                          where semester = 'Spring' and year = 2018)
```

- Find the total number of (distinct) students who have taken course taught by the instructor with *ID* 10101

```
select count (distinct ID)
from takes
where course_id in (select course_id from teaches
                       where teaches.ID = 10101);
```

takes relation

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>grade</i>
00128	CS-101	1	Fall	2017	A
00128	CS-347	1	Fall	2017	A-
12345	CS-101	1	Fall	2017	C
12345	CS-190	2	Spring	2017	A
12345	CS-315	1	Spring	2018	A
12345	CS-347	1	Fall	2017	A
19991	HIS-351	1	Spring	2018	B
23121	FIN-201	1	Spring	2018	C+
44553	PHY-101	1	Fall	2017	B-
45678	CS-101	1	Fall	2017	F
45678	CS-101	1	Spring	2018	R+

teaches relation

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017



Set Membership (Cont.)

- The **in** and **not in** operators can also be used on enumerated sets. For example, find all instructor names which are neither “Mozart” nor Einstein”

```
select name  
from instructor  
where name not in ('Mozart', 'Einstein')
```




Set Comparison – “some”

- Find names of instructors with salary greater than that of some (at least one) instructor in the Biology department.

```
select name
from instructor
where salary > some (select salary
                     from instructor
                     where dept_name = 'Biology');
```

- some** can be used with $<$, $\leq$, $>$, $=$, $\diamond$
- = some** is identical to **in**, whereas $\diamond$ **some** is *not* the same as **not in**.
For example:

“20 = **some** (10,20,30)” is true

“20 **in** (10,20,30)” is true

“20 $\diamond$ **some** (10,20,30)” is true

“20 **not in** (10,20,30)” is false



Set Comparison – “all”

- Find the names of all instructors whose salary is greater than the salary of all instructors in the Biology department.

```
select name
from instructor
where salary > all (select salary
                    from instructor
                    where dept name = 'Biology');
```

- all** can be used with <, ≤, >, =, ≠
- ◊ **all** is identical to **not in**, whereas **= all** is the *not* same as **in**. For example:

20 ◊ **all** (10,20,30) is false

20 **not in** (10,20,30) is false

20 = **all** (10,20,30) is false

20 **in** (10,20,30) is true

*Unlike the **in** operator, neither **some** nor **all** operators can be used on enumeration sets. So 'select name from instructor where salary > some(1000,20000)' is not allowed.*



Test for Empty Relations

- The **exists** operator returns the value **true** if the argument subquery is nonempty.
- **exists** $r \Leftrightarrow r \neq \emptyset$
- **not exists** $r \Leftrightarrow r = \emptyset$
- “Find all courses taught in both the Fall 2017 semester and in the Spring 2018 semester”

```
select distinct course_id
from section as S
where semester = 'Fall' and year = 2017 and
      exists (select *
              from section as T
              where semester = 'Spring' and year = 2018
              and S.course_id = T.course_id);
```

- A name from an outer query (e.g. S.course_id in the above query) can be used in a subquery. Such subquery is called a **correlated subquery**.



Use of “not exists”

- Find all students who have taken all courses offered in the Biology department.

```
select distinct S.ID, S.name  
from student as S  
where not exists ( (select course_id  
                    from course  
                    where dept_name = 'Biology')  
except  
                  (select T.course_id  
                   from takes as T  
                   where S.ID = T.ID));
```

- First nested query lists all courses offered in Biology
 - Second nested query lists all courses a particular student took
- Note that $X - Y = \emptyset \Leftrightarrow X \subseteq Y$
- Note: Cannot write this query using **= all** and its variants, for example:

```
select distinct ID from takes  
where course_id = all (select course_id from course  
                      where dept_name = 'Biology')
```





Test for Absence of Duplicate Tuples

- The **unique** operator tests whether a subquery has any duplicate tuples in its result.
- The **unique** operator evaluates to “true” if a given subquery contains no duplicates .
- Find all courses that were offered at most once in 2017

```
select T.course_id
from course as T
where unique ( select R.course_id
                  from section as R
                  where T.course_id= R.course_id
                      and R.year = 2017);
```



Scalar Subquery

- Scalar subquery is one which is used where a **single** value is expected. It can occur in **select**, **where**, and **having** clauses.
- List all departments along with the number of instructors in each department

```
select dept_name,  
      ( select count(*)  
        from instructor  
        where department.dept_name = dept_name)  
      as num_instructors  
from department;
```

Equivalent to:

```
select dept_name, count(*) from instructor group by dept_name
```

- List all instructors whose salary is greater than the average salary of their department

```
select name from instructor as I where salary >  
      ( select avg(salary) from instructor  
        where dept_name = I.dept_name)
```



Scalar Subquery (Cont.)

- Find student Zhang's advisor name.

```
select name from instructor  
where ID = ( select i_id from advisor  
             where s_id = ( select ID from student  
                           where name = 'Zhang'))
```

Equivalent to:

```
select I.name from student as S, advisor as A, instructor as I  
where S.ID=A.s_id and I.ID=A.i_id and S.name = 'Zhang'
```

instructor relation

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

student relation

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

advisor relation

s_id	i_id
00128	45565
12345	10101
23121	76543
44553	22222
45678	22222
76543	45565
76653	98345
98765	98345
98988	76766



Scalar Subquery (Cont.)

- List all departments whose instructor average salary is greater than the average salary of all other instructors

Can this query be written like this?

```
select dept_name from instructor as I1 where avg(salary)>
(select avg(salary) from instructor
where dept_name <> I1.dept_name)
```



```
select dept_name from instructor as I1
group by dept_name having avg(salary)>
(select avg(salary) from instructor
where dept_name <> I1.dept_name)
```



How to prove the result is correct?

```
select dept_name, avg(salary) as deptavg,
(select avg(salary) from instructor
where dept_name <> I1.dept_name) as otheravg
from instructor as I1 group by dept_name
```




Scalar Subquery (Cont.)

- If a subquery nested as a scalar subquery returns multiple result tuples, a runtime error occurs

select *name* **from** *instructor* **where** *salary* >=
(**select** *salary* **from** *instructor*);



select *name* **from** *instructor* **where** *salary* >=
(**select** *salary* **from** *instructor* **where** *ID* = 10101);





Subqueries in the From Clause

- SQL allows a subquery to be used in the **from** clause
- Find all departments whose average salary is greater than \$42,000.

select *dept_name*

from (**select** *dept_name*, **avg** (*salary*) **as** *avg_salary*

from *instructor* **group by** *dept_name*) **as** *dept_avg*

where *avg_salary* > 42000;

Equivalent to:

select *dept_name* **as** *avg_salary* **from** *instructor*

group by *dept_name* **having** **avg** (*salary*) > 42000;

Note that some SQL implementations, notably MySQL, **SQL Server** and PostgreSQL, require that each subquery relation in the **from** clause must be given a name, even if the name is never referenced;



With Clause

- Using nested subqueries may make the query harder to read and understand. The **with** clause may makes the query logic clearer
- Find all departments whose average salary is greater than \$42,000.

with *dept_sal* **as**

 (**select** *dept_name*, **avg** (*salary*) **as** *avg_salary*
 from *instructor* **group by** *dept_name*)

select *dept_name*, *avg_salary* **from** *dept_sal*

where *avg_salary* > 42000;

- The **with** clause provides a way of defining a **temporary relation** whose definition is available **only** to the query in which the **with** clause occurs.
 - corresponds to the **assignment** operation of the relational algebra



3.10 Modification of the Database

- **Deletion** of tuples from a given relation.
- **Insertion** of new tuples into a given relation
- **Updating** of values in some tuples in a given relation



Deletion

- SQL expresses a deletion by:

delete from r where P

- r represents a relation
- P is a predicate

- Delete all instructors

delete from *instructor*

- Delete all instructors from the Finance department

**delete from *instructor*
where *dept_name* = 'Finance';**

- Delete all instructors whose department located in the Watson building from the instructor relation.

**delete from *instructor*
where *dept_name* in (select *dept_name*
from *department*
where *building* = 'Watson');**



Insertion

- SQL expresses a insertion by:

insert into $r (A_1, A_2, \dots, A_n)$
values $(V_1, V_2, \dots, V_n)$

- r represents a relation
 - A_i represents an attribute
 - V_i represents a value
- Add a new tuple to *course*

insert into *course* (*course_id*, *title*, *dept_name*, *credits*)
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- or equivalently

insert into *course*
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

The values are specified in the order in which the corresponding attributes are listed in the relation schema.



Insertion (Cont.)

- Add multiple tuples to *course*

```
insert into course (course_id, title, dept_name, credits)  
  values ('CS-437', 'Database Systems', 'Comp. Sci.', 4),  
          ('CS-401', 'Data Structure', 'Comp. Sci.', 5),  
          ('CS-412', 'Compiler', 'Comp. Sci.', 3);
```

- Equivalent to

```
insert into course (course_id, title, dept_name, credits)  
  values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);  
  
insert into course (course_id, title, dept_name, credits)  
  values ('CS-401', 'Data Structure', 'Comp. Sci.', 5);  
  
insert into course (course_id, title, dept_name, credits)  
  values ('CS-412', 'Compiler', 'Comp. Sci.', 3);
```



Insertion (Cont.)

- Make each student in the Music department who has earned more than 144 credit hours an instructor in the Music department with a salary of \$18,000.

```
insert into instructor(ID, name, dept_name, salary)  
  select ID, name, dept_name, 18000  
  from student  
  where dept_name = 'Music' and tot_cred > 144;
```

- Evaluating the **select** statement completely before performing insertions.

Otherwise queries like

```
insert into table1 select * from table1
```

would cause problem



Updates

- SQL expresses a updating by:

update *r*

set $A_1 = V_1, A_2 = V_2, \dots, A_n = V_n$

where *P*

- *r* represents a relation
 - A_i represents an attribute
 - V_i represents a value
 - *P* is a predicate
- Give a 5% salary raise to all instructors

update *instructor*
set *salary* = *salary* * 1.05
 - Give a 5% salary raise to those instructors who earn less than 70000

update *instructor*
set *salary* = *salary* * 1.05
where *salary* < 70000;



Updates with Scalar Subqueries

- Give a 5% salary raise to instructors whose salary is less than average

```
update instructor
set salary = salary * 1.05
where salary < (select avg (salary)
                from instructor);
```

Evaluating the **select** statement completely before performing updates.

- Recompute and update tot_creds value for all students

```
update student
set tot_cred = (select sum(credits)
                from takes, course
                where takes.course_id = course.course_id and
                       takes.ID = student.ID and
                       takes.grade is not null and
                       takes.grade <> 'F');
```

course relation

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3

student relation

ID	name	dept_name	tot_cred
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46

takes relation

ID	course_id	sec_id	semester	year	grade
0128	CS-101	1	Fall	2017	A
0128	CS-347	1	Fall	2017	A-
2345	CS-101	1	Fall	2017	C
2345	CS-190	2	Spring	2017	A
2345	CS-315	1	Spring	2018	A
2345	CS-347	1	Fall	2017	A
9991	HIS-351	1	Spring	2018	B
3121	FIN-201	1	Spring	2018	C+



End of Chapter 3